

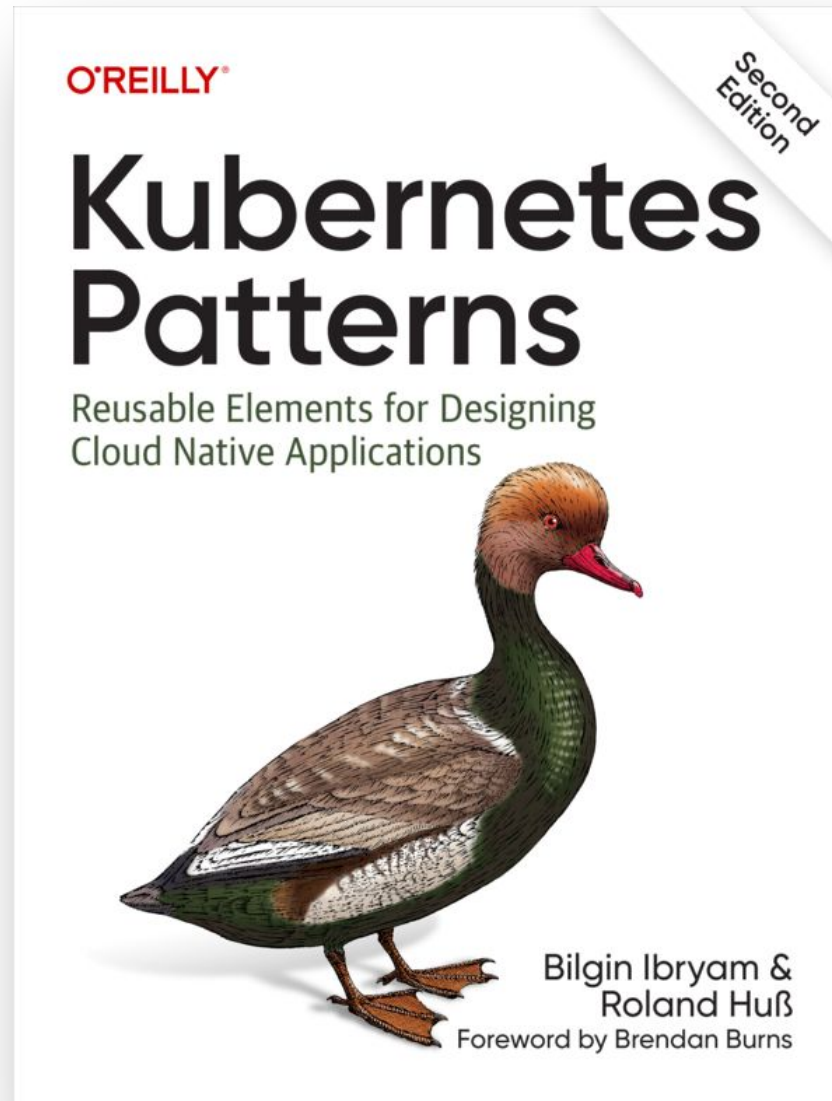


Hands-on Knative

JAX 2025

Dr. Roland Huß @ro14nd
Distinguished Engineer, Red Hat
Red Hat OpenShift AI
Knative TOC alumni





<https://k8spatterns.com>

Wait ... wat ?



Serverless vs. FaaS

Serverless is a **Deployment Model** that abstracts away the driving machine infrastructure.

- No server management required
- Executed, scaled and billed according to demand
- Defines a deployment packaging, but otherwise agnostic to the application

FaaS (Function-as-a-Service) is a **Programming Model** that mandates developing your application with fine grained function that match a given signature.

- Deployed as Serverless application
- Typically used as *glue code* to connect services

A photograph of a wooden boat's interior, looking towards the bow. The boat is made of weathered wood and has several coils of rope on the deck. In the background, the ocean stretches to a horizon with several large, rocky islands under a blue sky with scattered white clouds. The water is a deep blue with some white foam visible near the boat's hull.

Knative

Kubernetes-based platform to
**deploy and manage modern
serverless workloads.**

<https://knative.dev>

Components

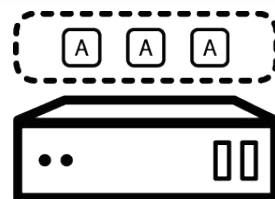
Function

Programming model that lets you focus on just your code for faster iterations.



Serving

A request-driven model that serves the container with your application and can "scale to zero".



Eventing

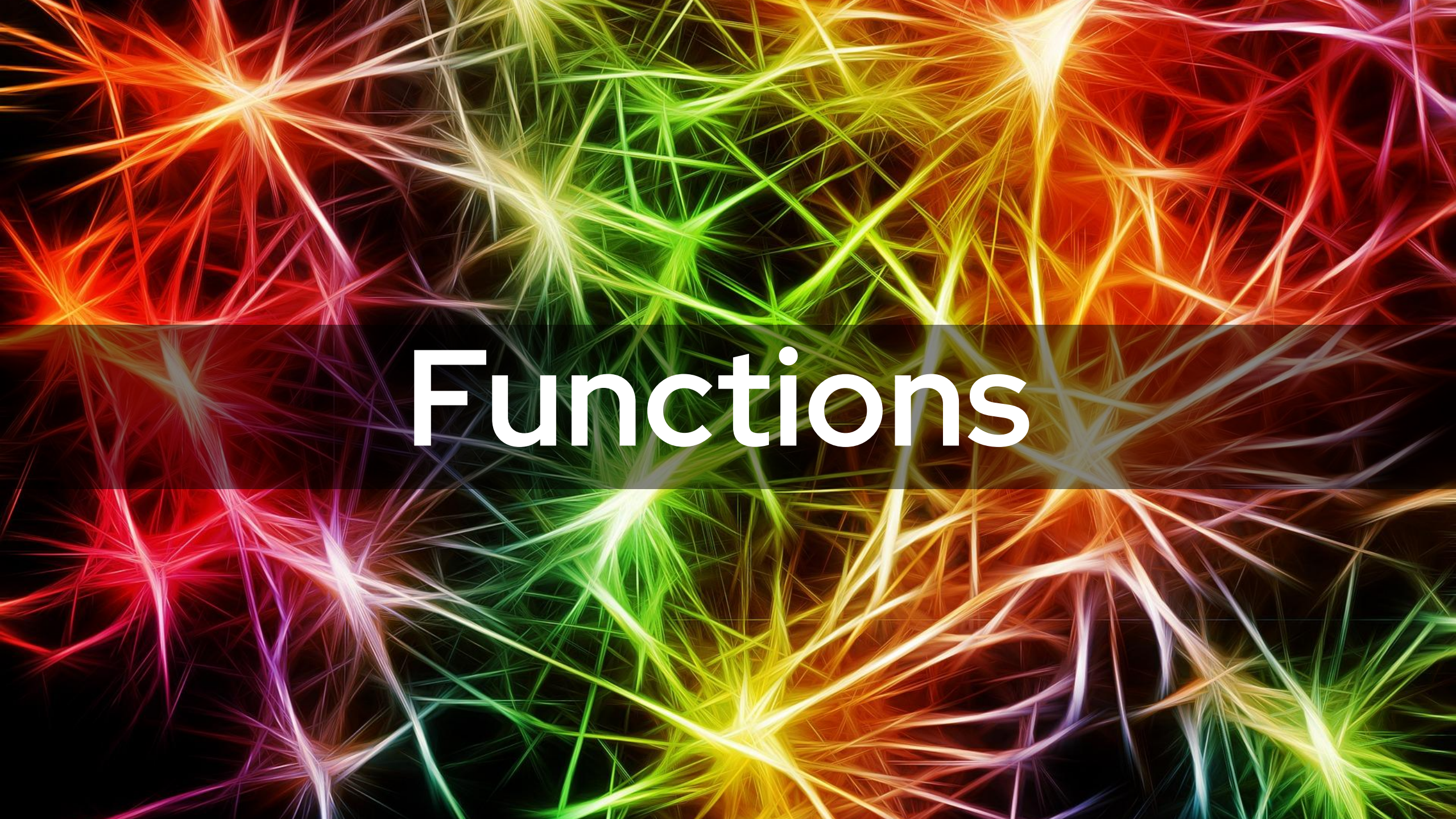
Common infrastructure for consuming and producing events that will stimulate applications.



Try Knative !

- Install from resource descriptors on Kubernetes Cluster
 - <https://knative.dev/docs/install/>
- Google **Cloud Run** (managed and on GKE)
 - <https://cloud.google.com/run/>
 - Not all Knative features implemented
 - see <https://ahmet.im/blog/cloud-run-is-a-knative>
- IBM Cloud **Code Engine**
 - <https://www.ibm.com/cloud/code-engine>
 - Vanilla Knative, additional workloads (like batch)
- Red Hat **OpenShift Serverless**
 - <https://www.openshift.com/learn/topics/serverless>
 - Supports all Knative features

Installation



Functions

func

- Opinionated **programming model** for Knative
 - Scaffolding of project templates for multiple runtimes
 - Quarkus, Node.js, Spring, Python, ...
 - [Funqy](#): Quarkus library for function programming (not only Knative)
 - Building and pushing container images
 - Deploying as Knative services
- Available **standalone** CLI or as **plugin** of the Knative CLI
 - <https://github.com/knative/func>
- **Local development** mode
- Technologies
 - Cloud-native Buildpacks or S2I
 - Local Docker/Podman or on-cluster builds

From Source to URL



Serving

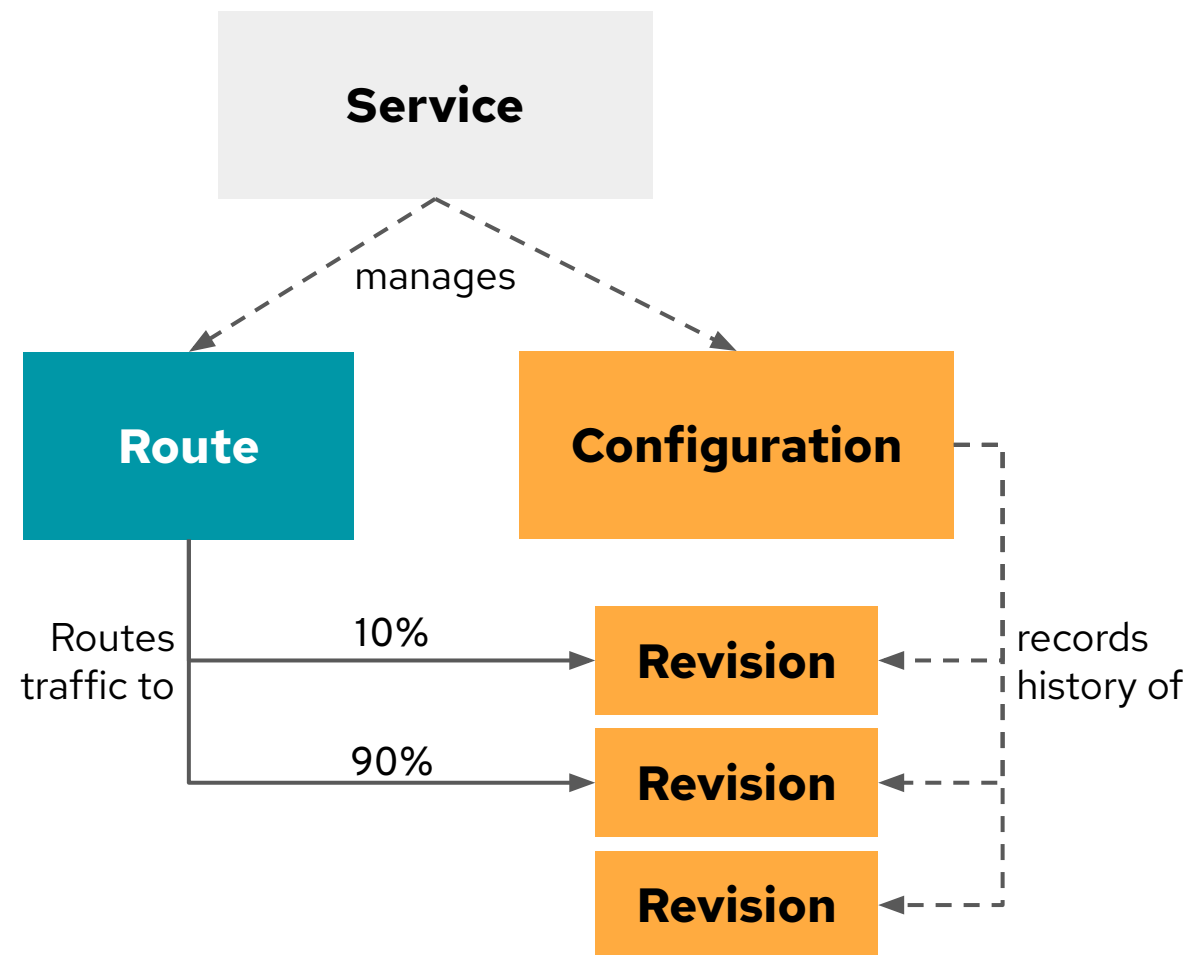
Route, scale-to-zero and track application revisions with ease.

Concepts

- **Demand-based autoscaling**, including scale-to-zero
- Separation of code and configuration
- Opinionated and simplified deployment model catered for **stateless applications**
- Rich **traffic split capabilities** to enable custom rollout strategies of new versions

Resources

- **Configuration** represent the *floating HEAD* of a history of **Revisions**
- **Revision** represents an immutable snapshot of code and configuration
- **Route** configure ingress over a collection of Revisions
- **Service** (not K8s services !) is a top-level entity that manage a set of Routes and Configurations



From Deployment to KService

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: random
spec:
  replicas: 1
  selector:
    matchLabels:
      app: random
  template:
    metadata:
      labels:
        app: random
    spec:
      containers:
        - image: rhuss/random
          name: random
          ports:
            - containerPort: 8080
```

```
apiVersion: serving.knative.dev/v1
kind: Service
metadata:
  name: random
spec:
  replicas: 1
  selector:
    matchLabels:
      app: random
  template:
    metadata:
      labels:
        app: random
    spec:
      containers:
        - image: rhuss/random
          name: random
          ports:
            - containerPort: 8080
```

No more K8s
Service or
Ingress/Route
required!

Autoscaling & Traffic Split

A scenic view of a city canal at sunset. The canal is flanked by tall, multi-story brick buildings with many windows. A bridge crosses the canal in the foreground. In the background, a modern skyscraper with a glass facade is visible against a golden sky. The word "Eventing" is overlaid in large white letters.

Eventing

**Universal subscription, delivery,
and management of CloudEvents.**

Eventing

- Based on CloudEvents (CNCF Standard)
- Pluggable event transport via **Channels**
 - In-Memory
 - Apache Kafka
 - Nats
 - RabbitMQ
- Flexible routing of events from Sources to Sinks
 - **Source**: Adapter for integrating 3rd party systems and emitting CloudEvents
 - **Sink**: Addressable endpoint for CloudEvents (like a Knative Service)



Event Sources

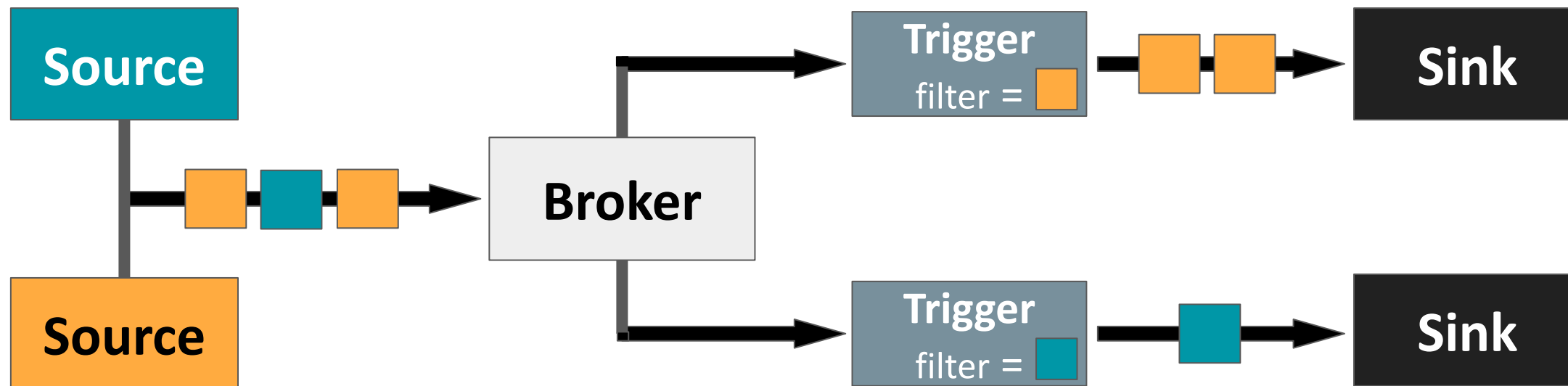
- Integrating 3rd party systems with Knative
- More often “**Adapter**” than an original event source
- Declared with a **Custom Resource**
- Evaluated by an Operator
- Push or Pull based
- Converting custom event formats to **CloudEvents**

Sources

Builtin Sources	
PingSource	Emitting static CloudEvents periodically
ApiServerSource	Kubernetes API Server events as CloudEvents
SinkBinding	Binds an arbitrary Pod specification to a Sink
ContainerSource	Meta-Source combining SinkBinding & Deployment
Contributed Sources	
GitHubSource	Converts GitHub webhooks events to CloudEvents
KafkaSource	Apache Kafka messages as CloudEvents
RabbitMQSource	Import RabbitMQ messages as CloudEvents

and many more: <https://knative.dev/docs/eventing/sources/>

Source → Service: Broker & Trigger



Broker

- Eventing Mesh for distributing Events
- Addressed by sources as sink

Trigger

- Filter on CloudEvent attributes (e.g. type)
- Connects a Sink with Broker

Source → Service: Broker & Trigger

- **Broker**

- Event Mesh (or Event Delivery System)
- Connects Sources with Sinks
- Uses Channels internally, creating on the fly
- Multi-tenant

- **Trigger**

- Filter events (e.g. type and/or source)
- Can produce new events (HTTP response is returned to Broker)
- Delivered as CloudEvents
- Dead-Letter Sinke for events that could not be processed

Broker & Triggers

A photograph of a wooden boat's interior, looking towards the bow. The boat is made of weathered wood and has several coils of rope on board. In the background, the ocean stretches to the horizon under a blue sky with scattered white clouds. Several small, rocky islands are visible in the distance. The word "Summary" is overlaid in a large, white, sans-serif font across the middle of the image.

Summary

Summary

Knative Function & Serving

- Simplified Deployment for stateless workloads
- Traffic based autoscaling including Scale-to-Zero
- Traffic splitting for custom rollout / rollback scenarios

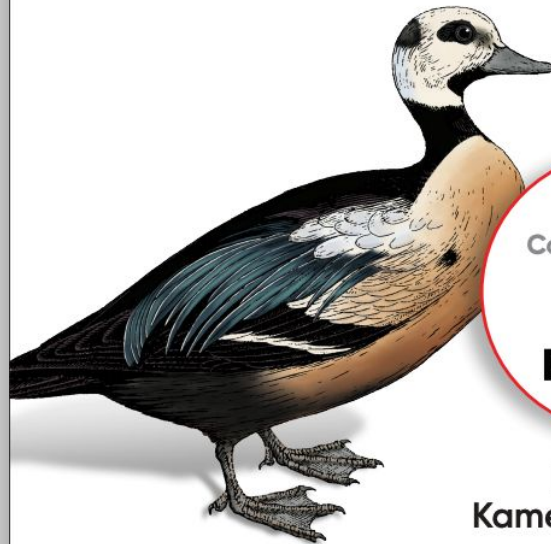
Knative Eventing

- External Triggers for feeding Knative Services
- Based on CloudEvents
- Backed by proven messaging systems
- Flexible messaging setup

O'REILLY®

Knative Cookbook

Building Effective Serverless Applications
with Kubernetes and OpenShift



Compliments of



Red Hat

Burr Sutter &
Kamesh Sampath

Thank you



ro14nd@hachyderm.io



<https://www.linkedin.com/in/ro14nd/>

Picture Credits

<https://www.pexels.com/photo/boat-island-ocean-sea-218999/>

<https://unsplash.com/photos/t6t2-gXKxXM>

<https://unsplash.com/photos/UGMf30W28qc>

<https://pixabay.com/photos/hamburg-speicherstadt-channel-2976711/>

<https://pixabay.com/photos/beer-machine-alcohol-brewery-1513436/>

<https://pixabay.com/photos/camel-sunset-landscape-tourism-2500618/>

<https://unsplash.com/photos/9SWHlgu8A8k>

<https://me.me/i/aws-lambda-is-just-glorified-cgi-bin-imgflip-com-change-my-mind-d0b715592ba34b08b79452ad02783ca2>

https://unsplash.com/photos/dodn_OTESN0

<https://pixabay.com/photos/annoy-cells-stars-dendrites-sepia-2926087/>